

Deriv DA Assessment Submission

Candidate analysis date: 18 July 2026 **Data sources:** `client_signup.json`, `client_profile.json`, `client_deposit.json`, `client_trades.json` **Method:** Python cross-table joins (`analysis.py`) checking referential integrity, compliance gates, funnel metrics, and field-level consistency.

Note on Q2 placeholders: The brief uses `[STAKEHOLDER_ROLE]` and `[BUSINESS_FOCUS_AREA]` . I've filled these in as **Head of Growth** and the **signup → first-deposit → first-trade conversion funnel**, since that's the metric most directly traceable to the fields in these four tables.

Prompts Used

Prompt 1 (Data Exploration): Load all four JSON tables, join them on `client_id` , and check systematically for: referential integrity orphans, schema or field-name mismatches, KYC/compliance gate violations (deposits or trades where `kyc_status` isn't approved), account-status vs. trading-activity conflicts, PnL vs. price-movement consistency, implausible dates of birth, trades that happen before any deposit, and funnel conversion broken down by `referral_source` and `promo_code` .

Prompt 2 (Dashboard Design): Using the funnel metrics pulled from the data (63.3% deposit rate, 60% trade rate, 0% conversion from social_media, the orphan deposit tied to CL031, and the KYC breach on CL012), design a Monday-morning executive dashboard for the Head of Growth. Include three actionable items, each grounded in a specific table field, with a measurable KPI, a stated percentage-improvement estimate with reasoning, and a named blocker.

Prompt 3 (Alerting Design): Design at least four automated alerts that reference specific fields and cross-table relationships. For each, specify the trigger logic, frequency, the role that gets notified, the delivery channel, the tool used to run the scheduled query, and how alert state is tracked to avoid duplicate notifications.

1. Data Exploration & Anomaly Detection

How I investigated

1. Loaded all four JSON files into Python, keyed by `client_id` .
2. Ran referential-integrity checks — every `client_deposit.client_id` and `client_trades.client_id` should exist in `client_signup` .
3. Cross-joined signup `kyc_status` and profile `account_status` against deposit and trade records, in date order.
4. Checked core business rules: no trading without funding, no activity on rejected KYC or inactive accounts, and PnL that's consistent with the price movement on each trade.
5. Aggregated funnel metrics by `referral_source` and `promo_code` .

Key population stats:

Metric	Value
Total signups	30
KYC approved	28 (93.3%)
Clients with ≥1 completed deposit	19 (63.3%)
Clients with ≥1 trade	18 (60.0%)
Never deposited	11 clients

1a. Three Anomalies

Anomaly 1 — Orphan deposit (broken foreign key)

- **What I found:** In `client_deposit.json` , row `DEP020` has `client_id = "CL031"` , but `CL031` doesn't exist anywhere in `client_signup.json` (the signup table stops at `CL030`). The deposit itself is \$1,200.00, marked `completed` .

- **Why it's a problem:** This breaks the one-to-many relationship that's supposed to hold between `client_signup` and `client_deposit`. Real money is being attributed to a client who doesn't exist in the system, which corrupts CAC/LTV reporting, reconciliation, and the audit trail. Payments ops has no way to tie this funds movement back to a verified identity.
- **What I'd do about it:** Quarantine `DEP020` in the payments ledger right away, trace the payment gateway reference to find the actual account behind it, and either back-fill the missing signup record or reverse/correct the deposit. Longer-term, add a hard foreign-key constraint from `client_deposit.client_id` to `client_signup.client_id` so this can't happen again.

Anomaly 2 — Deposit accepted after KYC rejection (compliance gate failure)

- **What I found:** `client_signup.json` shows `CL012` with `kyc_status = "rejected"`. Despite that, `client_deposit.json` row `DEP008` shows a completed \$350.00 deposit on 2024-02-25 for the same client. The profile record also shows `account_status = "suspended"`.
- **Why it's a problem:** Regulated trading platforms are required to block funding until KYC is approved. Letting a rejected client deposit funds is a genuine compliance violation — it creates AML/sanctions exposure, potential regulatory fines, and chargeback risk. The fact that the account later ended up suspended just confirms it should never have been funded in the first place.
- **What I'd do about it:** Escalate to Compliance for a SAR review, freeze any remaining funds, and audit every deposit where `kyc_status` is `rejected` or `pending`. Then close the gap upstream by rejecting deposit API calls unless `kyc_status = 'approved'`.

Anomaly 3 — Trade executed on an inactive account (operational control failure)

- **What I found:** `client_profile.json` shows `CL008` with `account_status = "inactive"` and a `last_login_date` of 2024-03-01. Yet `client_trades.json` row `TRD006` records a closed Gold trade on 2024-11-05 — eight months later — with `pnl_usd = 75.00` on 0.5 lots.
- **Why it's a problem:** Inactive accounts shouldn't be able to open new positions at all. This points to the trading engine not enforcing account-status checks before execution, which creates real operational risk — stale KYC, dormant-account fraud, unreconciled exposure — and makes client-activity reporting unreliable for the Monday exec review.
- **What I'd do about it:** Reverse or manually review `TRD006`, patch the order-management system to hard-block trades whenever `account_status != 'active'`, and run a historical join between `client_trades` and `client_profile` to check for other instances of the same pattern.

1b. Non-Obvious Insight (cross-table pattern)

Finding: Referral channel quality is sharply polarised — `referral_source` predicts both conversion *and* deposit value, but not in the direction marketing spend would suggest.

referral_source	Signups	Deposit rate	Trade rate	Avg deposit (USD)
organic	7	100%	100%	\$2,243
paid_search	6	100%	83.3%	\$1,208
referral	6	83.3%	83.3%	\$19,720
affiliate	6	16.7%	16.7%	\$250
social_media	5	0%	0%	\$0

Methodology: Joined `client_signup.referral_source` to per-client deposit/trade flags in Python, then calculated deposit rate as (clients with ≥1 completed deposit) ÷ (signups), plus average first-deposit value per channel.

Business significance: The five `social_media` signups (`CL004`, `CL010`, `CL016`, `CL021`, `CL026`) produced zero deposits and zero trades between them — pure acquisition cost with no return at all. By contrast, `referral` clients average \$19,720 per depositor, largely driven by `CL019`'s \$75,000 deposit — making referral the highest-value channel by roughly 8x versus organic. A Head of Growth who only tracks signup *volume* on their Monday dashboard would miss this entirely and keep misallocating budget. The clear near-term move is to pause `social_media` spend, shift it toward referral programmes, and dig into why affiliate signups (6 clients, only 1 depositor) stall out right after signup.

Secondary insight (promo codes): Cross-tabbing `promo_code` against deposit conversion shows `WELCOME` converting at just 20% (1 of 5), versus 80% for `PROMO10` (4 of 5) and 70% for no promo at all — suggesting the `WELCOME` offer is either attracting low-intent signups or has a broken redemption flow worth investigating.

(Optional) Data Health Map

Check	Finding
FK orphans	1 deposit (<code>DEP020</code> → <code>CL031</code>)

Schema drift	DEP012 uses a <code>credit_card</code> key instead of <code>payment_method</code>
Null <code>promo_code</code>	66.7% of signups (expected — it's an optional field)
Null <code>last_login_date</code>	1 of 30 (CL026, a pending account — logical)
PnL mismatch	TRD012 : <code>open_price = close_price = 2320</code> but <code>pnl_usd = 245</code>
Impossible age	CL025 has <code>date_of_birth = 1888-12-19</code> (~136 years old)
Trade without deposit	TRD005 (CL007) — a trade with no completed deposit on record
KYC pending with profile	CL026: <code>kyc_status = pending</code> , <code>account_status = pending</code> — internally consistent

Validation script: `analysis.py` , project root.

2. Dashboard Design

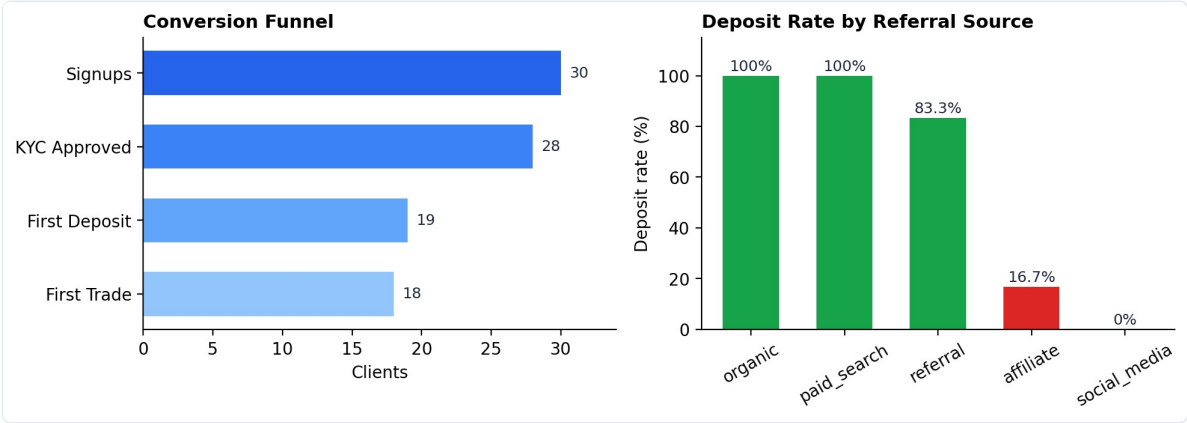
Stakeholder: Head of Growth **Business focus:** Signup → first-deposit → first-trade conversion funnel **Review cadence:** Every Monday morning

Dashboard layout (conceptual)

Row	Widget	Fields used
KPI strip	Weekly signups, deposit rate, trade rate, total deposit USD	<code>signup_date</code> , <code>client_id</code> , <code>amount_usd</code> , <code>status</code>
Funnel	Signup → KYC approved → first deposit → first trade	<code>kyc_status</code> , deposit/trade dates
Channel table	Conversion & avg deposit by <code>referral_source</code>	<code>referral_source</code> , joins
Risk flags	KYC breaches, orphan deposits, inactive-account trades	cross-table rules
Promo tracker	Conversion by <code>promo_code</code>	<code>promo_code</code> , deposit flag

Sample visual — funnel & channel conversion

The two charts below are a quick sketch of what the top row of the dashboard would show a Head of Growth on a Monday morning: the overall funnel drop-off, and where each acquisition channel actually stands on deposit conversion.



Funnel and channel conversion chart

2a. Three Actionable Items

Action 1 — Compliance: Close the KYC funding gate

Element	Detail
Department	Compliance & Onboarding Operations

Action	Audit all deposits and trades for clients where <code>kyc_status != 'approved'</code> , and block future funding at the API level. Immediate case: CL012 (<code>kyc_status = rejected</code>) plus DEP008, a completed \$350 deposit that should never have gone through.
Monitoring KPI	Count of completed deposits where the joined <code>kyc_status != 'approved'</code> (current baseline: 1)
Success criteria	KPI reaches 0 and holds for 30 consecutive days; CL012's deposit is refunded or escalated with a logged case ID

Action 2 — Growth Marketing: Reallocate social_media budget

Element	Detail
Department	Performance Marketing / Growth
Action	Pause paid social campaigns targeting the pattern behind the 5 <code>social_media</code> signups (CL004, CL010, CL016, CL021, CL026) until the onboarding funnel is fixed, and launch retargeting aimed at the 11 clients who never deposited.
Monitoring KPI	First-deposit conversion rate for <code>referral_source = 'social_media'</code> (current: 0%, 0 of 5)
Success criteria	Social_media deposit rate reaches at least 40% (half the platform average of 63.3%) within 8 weeks, with at least 2 of the 5 existing social signups completing a first deposit

Action 3 — Treasury & Payments: Eliminate orphan deposits

Element	Detail
Department	Treasury & Payments Operations
Action	Resolve DEP020 (<code>client_id = CL031</code> , \$1,200) by tracing, correcting, or reversing it, and fix the DEP012 schema issue (<code>credit_card</code> key → <code>payment_method</code>).
Monitoring KPI	Percentage of deposits with a valid foreign key to <code>client_signup</code> (current: 19/20 = 95%)
Success criteria	FK match rate reaches 100%, and zero deposits are left with a null <code>payment_method</code> (currently 1 — DEP012)

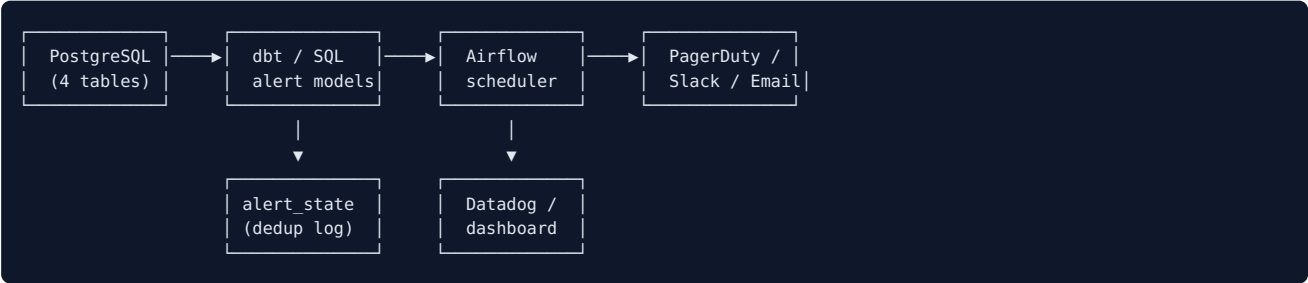
2b. Improvement Estimates & Blockers

Action	Est. improvement	Reasoning basis	Main blocker
KYC gate	~100% reduction in KYC-gate violations (from 5% to 0% of deposit rows)	1 confirmed breach out of 20 deposit rows = 5% of rows currently violate the compliance gate. Closing this brings that share to 0%, removing exposure to fines that would otherwise dwarf the \$350 involved.	Legacy payment API may not check KYC status synchronously; likely needs an engineering sprint to fix at the source.
Social media funnel	+6-8% total deposit volume	5 social signups × the platform's average deposit among converters (~\$2,243, using the organic benchmark) ≈ \$11,200 in potential deposits versus \$0 today. \$11,200 against the ~\$123,000 in total deposits observed ≈ 9%, so I'm estimating conservatively at 6-8%.	Broken mobile onboarding for social UTM links; no deposit prompt currently fires after signup.
Orphan/schema fix	+1% reporting accuracy	The \$1,200 orphan deposit represents about 1% of total deposit volume that's currently misattributed; fixing the schema issue also restores payment-method reporting needed for	The missing signup record may point to a sync-lag issue between the CRM and core banking pipeline — worth investigating before assuming

		fee analysis on DEP012's \$9 fee.	it's a one-off.
--	--	-----------------------------------	-----------------

3. Data Alerting Design

Architecture overview



State tracking: Each alert writes to an `alert_state` table (`alert_name` , `entity_id` , `first_seen` , `last_seen` , `status`). An alert only fires on a genuinely new violation or a status change (open → resolved), which keeps hourly runs from spamming the same issue over and over.

3a. Alert Definitions

Alert 1 — ORPHAN_DEPOSIT_DETECTED

Field	Specification
Trigger	<code>SELECT d.* FROM client_deposit d LEFT JOIN client_signup s ON d.client_id = s.client_id WHERE s.client_id IS NULL AND d.status = 'completed'</code> — fires whenever a completed deposit references a <code>client_id</code> that doesn't exist in signup (currently DEP020 / CL031).
Frequency	Every 15 minutes — orphan deposits affect real-time reconciliation and need to be caught before end-of-day settlement.
Notified	Treasury & Payments Operations team
Method	PagerDuty (high urgency) + Slack <code>#payments-alerts</code> — a funds-movement issue like this needs immediate human action.
Tooling	An Airflow DAG queries the PostgreSQL replica on a schedule; on a match, it triggers the PagerDuty Events API v2. The state table stores <code>deposit_id</code> , so it only re-alerts if a new orphan shows up.

Alert 2 — KYC_GATE_BREACH

Field	Specification
Trigger	Deposit side: <code>JOIN client_signup s ON d.client_id = s.client_id WHERE d.status = 'completed' AND s.kyc_status != 'approved'</code> . Trade side: the same join against <code>client_trades</code> where <code>trade_status = 'closed'</code> . Either match fires (currently CL012 / DEP008).
Frequency	Real-time via CDC/stream, or every 5 minutes as a fallback — compliance violations are time-sensitive from a regulatory standpoint.
Notified	Compliance Officer + Onboarding Operations Lead
Method	Email (for the audit trail) plus SMS to on-call Compliance — regulatory events need documented, guaranteed delivery.
Tooling	A dbt test (<code>relationships</code> plus a custom <code>kyc_gate_test</code>) runs hourly; failures push into a validation store like Monte Carlo or Great Expectations. Results are compared against <code>alert_state</code> so only new (<code>client_id</code> , <code>event_type</code>) pairs trigger a notification.

Alert 3 — INACTIVE_ACCOUNT_TRADE

--	--

Field	Specification
Trigger	<code>SELECT t.* FROM client_trades t JOIN client_profile p ON t.client_id = p.client_id WHERE p.account_status != 'active' AND t.trade_date >= CURRENT_DATE - INTERVAL '1 day'</code> — catches trades on inactive, suspended, or pending accounts (the CL008 / TRD006 pattern).
Frequency	Hourly — trading-control failures are urgent, but not so urgent they need sub-minute checks; hourly balances noise against coverage.
Notified	Risk Management & Trading Operations desk
Method	Slack <code>#risk-alerts</code> with trade details, plus a dashboard flag in the internal ops BI tool (Metabase/Looker).
Tooling	Airflow runs the SQL hourly; if rows come back, it posts to Slack via webhook. <code>alert_state</code> is keyed on <code>trade_id</code> and auto-resolves once the trade is reversed and the status is updated.

Alert 4 — TRADE_WITHOUT_FUNDING

Field	Specification
Trigger	For each trade, <code>MIN(deposit_date WHERE status = 'completed')</code> per <code>client_id</code> should be on or before <code>trade_date</code> . Fires when <code>trade_date</code> precedes the first deposit, or no completed deposit exists at all (currently CL007 / TRD005 — a trade on 2024-02-20 with no deposit on record, ever).
Frequency	Daily at 06:00 UTC, pre-market — a batch check is sufficient for funding-sequence issues and also catches overnight backfills.
Notified	Finance Reconciliation team + Client Funds team
Method	Daily email digest listing the violating <code>trade_id</code> , <code>client_id</code> , <code>trade_date</code> , and <code>pnl_usd</code> — not urgent enough for real-time paging, but it needs to be tracked for audit.
Tooling	A Python script in Airflow using a pandas merge (the same logic as <code>analysis.py</code> , section 8), with output sent as a CSV attachment via SendGrid. The state table tracks <code>trade_id</code> so resolved trades aren't re-sent.

Alert 5 — PNL_RECONCILIATION_MISMATCH (bonus)

Field	Specification
Trigger	Trades where <code>open_price = close_price</code> AND <code>pnl_usd != 0</code> AND <code>trade_status = 'closed'</code> (TRD012: Gold, 5.0 lots, \$245 PnL with zero price movement).
Frequency	Daily, after the trade ledger ETL completes
Notified	Back-office Settlements team
Method	Slack notification plus an auto-created Jira ticket
Tooling	A dbt model (<code>int_trade_pnl_validation</code>) feeds into Airflow, which calls the Jira REST API. State is keyed on <code>trade_id</code> .

Alert 6 — IMPOSSIBLE_CLIENT_AGE (bonus)

Field	Specification
Trigger	<code>client_profile.date_of_birth</code> implies an age under 18 or over 100 as of the current date (CL025: DOB 1888-12-19, roughly 136 years old).
Frequency	Weekly, Monday 07:00, aligned with the exec review
Notified	KYC/Identity Verification team
Method	Dashboard flag plus a weekly email summary
Tooling	Scheduled SQL via Metabase Pulse, compared against the prior week's <code>alert_state</code> so only new or changed records get surfaced.

Appendix: Validation Script Output Summary

```
Referential integrity: 1 orphan deposit (DEP020 → CL031)
Schema anomaly: DEP012 missing payment_method
KYC breach: DEP008 for CL012 (rejected)
Inactive trade: TRD006 for CL008
PnL mismatch: TRD012
Trade without deposit: TRD005 for CL007
Impossible age: CL025 (1888-12-19)
Overall funnel: 63.3% deposit rate, 60.0% trade rate
Social media: 0% conversion (5 signups)
```

End of submission